

PredictiveEdge System Architecture v1.0

Version: 1.0

Status: Draft for Review

Owner: PredictiveEdge Architecture Team

Last Updated: July 2026

1. Purpose

This document defines the high-level system architecture of PredictiveEdge.

It serves as the primary architectural reference for all contributors and establishes the boundaries, responsibilities, and interactions between major components of the platform.

All future architecture decisions, modules, APIs, and plugins should align with this document.

2. Product Vision

PredictiveEdge is an **Open Source AI Trading Intelligence Platform**.

It is **not a broker** and **not a trading exchange**.

Instead, it provides an intelligent decision layer above existing brokers to help traders make better decisions before, during, and after every trade.

3. Primary Goals

The platform aims to:

- Deliver AI-assisted trade analysis.
 - Support multiple brokers through a common abstraction layer.
 - Remain broker-independent.
 - Provide explainable AI rather than black-box recommendations.
 - Enable both retail and professional traders.
 - Build a community-driven open-source ecosystem.
-

4. Core Principles

4.1 Broker Independence

PredictiveEdge must never depend on a specific broker.

All broker integrations must be implemented as plugins using the Broker Adapter Framework.

4.2 AI is Advisory by Default

AI assists traders by analysing opportunities and risk.

The trader remains in control of execution.

Optionally, users may enable AI-assisted execution through configurable settings.

4.3 Explainable Decisions

Every AI recommendation must include:

- Reasoning
 - Confidence Score
 - Supporting indicators
 - Risk analysis
 - Suggested stop-loss levels
 - Suggested target levels
-

4.4 Event-Driven Design

Market events, broker events, AI events, and portfolio events are exchanged through an internal event model to keep modules loosely coupled.

4.5 Plugin First

Every external integration should be replaceable.

Examples include:

- Brokers
- AI Providers
- Market Data Providers
- Notification Providers
- Authentication Providers

5. High-Level Architecture

The platform consists of six logical layers:

1. Presentation Layer
2. Application Layer
3. Trading Intelligence Layer
4. Trade Execution Gateway
5. Broker Adapter Framework
6. Infrastructure Layer

Each layer has a clearly defined responsibility and communicates only through well-defined interfaces.

6. Layer Responsibilities

6.1 Presentation Layer

Provides user interaction.

Responsibilities:

- Web application
- Mobile application (future)
- REST API
- WebSocket updates
- User dashboards

Technology (initial):

- React
 - TypeScript
 - Progressive Web App
-

6.2 Application Layer

Coordinates user-facing business capabilities.

Responsibilities:

- Authentication
- User Profiles
- Portfolio
- Watchlists
- Alerts
- Notifications

- Workspace Management
- User Settings

This layer does not perform AI analysis or broker communication directly.

6.3 Trading Intelligence Layer

This is the core of PredictiveEdge.

Responsibilities include:

- AI Decision Engine
- Strategy Engine
- Risk Engine
- Scanner Engine
- Indicator Engine
- Pattern Recognition
- News & Sentiment Analysis
- Portfolio Intelligence
- Backtesting
- Explainable AI
- Trade Planner
- Trade Passport Generation

No direct broker communication occurs in this layer.

6.4 Trade Execution Gateway

Acts as the safety boundary between AI recommendations and broker execution.

Responsibilities:

- Validate orders
- Apply user preferences
- Apply risk limits
- Calculate position size
- Generate Stop Loss (SL1, SL2, SL3)
- Generate Targets (T1, T2, T3)
- Create Bracket/OCO orders (where supported)
- Prevent duplicate execution
- Record execution audit

Every trade passes through this gateway.

6.5 Broker Adapter Framework

Provides a common interface for all supported brokers.

Initial adapters:

- FYERS
- Zerodha
- Groww
- Upstox
- Angel One
- Paper Trading

Future brokers can be added without changing the platform core.

6.6 Infrastructure Layer

Provides shared technical capabilities.

Components include:

- PostgreSQL
- Redis
- Object Storage
- Event Streaming
- AI Model Runtime (Ollama)
- Cloud AI Providers
- Logging
- Monitoring
- Scheduler

Infrastructure services must remain independent from business logic.

7. External Systems

PredictiveEdge interacts with:

- Broker APIs
- Market Data Providers
- Stock Exchanges
- News Providers
- AI Providers
- Email Services
- SMS Providers
- Push Notification Services

These integrations are isolated through adapters.

8. Trade Lifecycle

A trade progresses through the following stages:

1. Market Observation
2. Opportunity Detection
3. AI Analysis
4. Risk Assessment
5. Trade Plan Generation
6. User Review (optional based on settings)
7. Broker Execution
8. Trade Monitoring
9. Exit Recommendation
10. Post-Trade Analysis
11. Continuous Learning

This lifecycle forms the foundation for future automation.

9. Architectural Constraints

The following constraints apply to all modules:

- No broker-specific logic in the core platform.
 - No AI provider-specific logic in business modules.
 - Business logic must not depend on UI frameworks.
 - Modules communicate through interfaces and events.
 - Components should remain independently testable.
-

10. Technology Direction

Initial technology choices:

- Java 21
- Spring Boot
- Spring AI
- React
- PostgreSQL
- Redis
- Kafka (planned)
- Ollama (default local AI)
- OpenAI (optional)
- Docker
- Kubernetes (future)

These technologies may evolve without changing the architectural principles.

11. Future Evolution

Future versions of the architecture will introduce:

- Plugin Marketplace
- Strategy Marketplace
- AI Model Marketplace
- Multi-user collaboration
- Mobile applications
- Cloud-native deployment
- Distributed event processing

The layered architecture defined in this document is intended to support these future capabilities without requiring major redesign.

12. Conclusion

The primary objective of PredictiveEdge is to provide a broker-independent, AI-driven trading intelligence platform that empowers traders with transparent, explainable, and extensible decision support.

This document establishes the architectural foundation upon which all future modules, integrations, and engineering decisions will be built.

Subsequent documents will define the system context, component interactions, event model, deployment architecture, and implementation details.